

Detecção Não Supervisionada de Anomalias em Séries Temporais Financeiras Brasileiras com LSTM-Autoencoder

Relatório Preliminar

Ana Beatriz and Miguel Cerne

Redes Neurais (2026.1)

22 de junho de 2026

Resumo

A detecção de anomalias em mercados financeiros é dominada, na literatura, pelo mercado norte-americano e por criptomoedas. Este trabalho aplica uma arquitetura *LSTM-Autoencoder* à detecção não supervisionada de anomalias em quatro ações da B3 (PETR4, VALE3, AMER3 e ITUB4), cobrindo distintos setores econômicos. O modelo é treinado apenas sobre o período considerado de “normalidade” (2010–2019) e sinaliza como anômalas as janelas cujo erro de reconstrução excede um limiar; comparam-se limiares estático (percentil fixo) e dinâmico (percentil em janela móvel causal). A avaliação combina validação narrativa (sobreposição com eventos econômicos e políticos brasileiros documentados) e quantitativa (injeção controlada de anomalias sintéticas, com Precision, Recall e F1). Este relatório preliminar documenta, de forma incremental e academicamente rastreável, as decisões metodológicas, suas fontes e as observações de cada etapa do projeto. Até o momento concluíram-se as etapas de *infraestrutura* (M0) e de *coleta e análise exploratória* (M1).

Palavras-chave: detecção de anomalias; LSTM-Autoencoder; séries temporais financeiras; B3; aprendizado não supervisionado.

Sumário

1	Introdução	2
1.1	Contribuições	2
1.2	Ativos e período	2
1.3	Sobre este documento	3
2	Trabalhos Relacionados	3
3	Metodologia	3
3.1	Pré-processamento	3
3.2	Modelo	3
3.3	Detecção	3
3.4	Avaliação	4
4	M0 — Infraestrutura e Reprodutibilidade	4
4.1	Ambiente e dependências	4
4.2	Configuração centralizada e seeds	4
4.3	Decisões de hiperparâmetros e proveniência	4
4.4	Riscos metodológicos registrados	4

5	M1 — Coleta e Análise Exploratória	5
5.1	Coleta e cache	5
5.2	Integridade das séries	5
5.3	Anomalias reais identificadas	5
5.4	O caso AMER3	6
6	M2 — Pré-processamento	6
6.1	Log-retornos	6
6.2	Split temporal e normalização	6
6.3	Revisão da hipótese sobre o AMER3	6
7	Conclusões Parciais e Próximos Passos	7

1 Introdução

A identificação de comportamentos atípicos em séries de preços de ativos é relevante tanto para a gestão de risco quanto para a compreensão de como eventos econômicos e políticos se refletem nos mercados. A maior parte da literatura de detecção de anomalias com aprendizado profundo concentra-se no mercado norte-americano e em criptomoedas [3, 4], deixando sub-representado o contexto de mercados emergentes como o brasileiro.

Este trabalho investiga a detecção *não supervisionada* de anomalias em ações da B3 por meio de um *LSTM-Autoencoder* [2]. A premissa é que um autoencoder treinado apenas em períodos de comportamento típico aprende a reconstruir os padrões normais da série; diante de movimentos atípicos, o erro de reconstrução cresce e sinaliza a anomalia.

1.1 Contribuições

O projeto combina quatro contribuições:

1. Aplicação a ativos nacionais (B3), contexto sub-representado na literatura.
2. Correlação sistemática e documentada entre anomalias detectadas e eventos históricos brasileiros.
3. Comparação entre limiar *estático* (percentil fixo) e *dinâmico* (percentil em janela móvel).
4. Avaliação de generalização entre setores econômicos distintos.

1.2 Ativos e período

Foram selecionados quatro ativos, um por setor, conforme a Tabela 1. O período de coleta vai de 2010 a 2024; o treino (“normalidade”) usa 2010–2019 e o teste, 2020–2024.

Tabela 1: Ativos analisados.

Ticker	Empresa	Setor
PETR4.SA	Petrobras PN	Energia/commodities
VALE3.SA	Vale S.A.	Mineração
AMER3.SA	Americanas S.A.	Varejo
ITUB4.SA	Itaú Unibanco	Financeiro

1.3 Sobre este documento

Trata-se de um relatório preliminar e *vivo*: a cada fechamento de *milestone* acrescenta-se uma seção descrevendo, academicamente, as decisões tomadas, suas fontes, as observações empíricas e as conclusões parciais. As decisões de configuração são registradas em paralelo como *Architecture Decision Records* (ADRs) no repositório (`docs/adr/`), referenciados ao longo do texto.

2 Trabalhos Relacionados

A detecção de anomalias com autoencoders recorrentes baseia-se na ideia de que a reconstrução de padrões aprendidos falha sobre observações fora da distribuição de treino. Li [4], em referência amplamente reproduzida, aplica um LSTM-Autoencoder a preços de ações (Johnson & Johnson), adotando janelas de 30 passos de tempo e definindo o limiar de anomalia a partir do erro de reconstrução do treino. Valkov [7] fornece uma implementação canônica em Keras, com escolhas de projeto que servem de referência prática (camadas LSTM de 64 unidades, *dropout* de 0,2, *validation_split* de 0,1 e, crucialmente, *shuffle=False* para preservar a ordem temporal). Petrovic [6] disponibiliza um repositório com abordagem equivalente.

Trabalhos mais recentes ampliam o desenho experimental. Liu et al. [5] propõem um arcabouço híbrido que acopla um LSTM-Autoencoder a redes adversárias generativas e introduzem explicitamente um *mecanismo de injeção artificial de anomalias* para avaliação quantitativa, validado em múltiplas categorias de ações — desenho que inspira diretamente a avaliação setorial e por injeção adotada neste projeto. No domínio de criptomoedas, IEEE [3] exploram a mesma família de modelos sobre valores do Bitcoin.

Em relação a esses trabalhos, a principal lacuna endereçada aqui é a aplicação ao mercado brasileiro (B3) com correlação documentada a eventos nacionais e comparação explícita entre estratégias de limiar estático e dinâmico.

3 Metodologia

Esta seção descreve o desenho metodológico do projeto. Os valores numéricos de hiperparâmetros e suas justificativas são apresentados nas seções de cada *milestone* e consolidados nos ADRs do repositório.

3.1 Pré-processamento

A feature principal é o *retorno logarítmico diário*, $r_t = \ln(P_t/P_{t-1})$, sobre o preço de fechamento ajustado. Adota-se um **split temporal antes da normalização**: o `MinMaxScaler` é ajustado *apenas* sobre o treino e então aplicado ao teste, evitando vazamento de informação do futuro. A série é então segmentada em *janelas deslizantes* de tamanho fixo (ver Seção 5).

3.2 Modelo

A arquitetura é um *LSTM-Autoencoder* [2]: um codificador LSTM comprime a janela em uma representação latente (gargalo), e um decodificador LSTM reconstrói a sequência, otimizando o erro quadrático médio (MSE) de reconstrução. Treina-se **um modelo por ativo**, condição necessária para a comparação setorial.

3.3 Detecção

Uma janela é marcada como anômala quando seu erro de reconstrução ultrapassa um limiar. Comparam-se duas estratégias:

- **Estático:** percentil fixo (95) do erro de reconstrução do treino.

- **Dinâmico:** percentil em janela móvel *causal* (apenas o passado), sensível a mudanças de regime.

3.4 Avaliação

Por ser um problema não supervisionado, a avaliação combina duas vias [5]:

- **Narrativa:** verificação de que as anomalias detectadas correspondem a eventos reais conhecidos (e.g., *crash* de março/2020; caso Americanas em janeiro/2023).
- **Quantitativa:** injeção controlada de anomalias sintéticas (*price shocks* e *volume spikes*) em posições conhecidas, permitindo calcular Precision, Recall e F1.

Nota metodológica. O período de treino (2010–2019) contém eventos de forte volatilidade (recessão de 2014–2016, Operação Lava Jato, *impeachment* de 2016). A definição de “normalidade” é, portanto, relativa, e essa limitação é assumida explicitamente.

4 M0 — Infraestrutura e Reprodutibilidade

A primeira etapa estabeleceu a infraestrutura que garante reprodutibilidade e separação entre lógica e orquestração: a lógica reside em `src/` e os notebooks apenas orquestram e visualizam.

4.1 Ambiente e dependências

Fixou-se o interpretador em **Python 3.12.13** (via `pyenv`), versão idêntica à do Google Colab à época, alvo principal de execução. As dependências foram *pinadas* (TensorFlow/Keras [1], `yfinance`, `pandas`, `numpy`, `scikit-learn`, `matplotlib`, `seaborn`), habilitando instalação editável (`pip install -e .`). Notebooks têm suas saídas removidas automaticamente no versionamento (`nbstripout`), reduzindo ruído em *diffs*.

4.2 Configuração centralizada e seeds

Todos os hiperparâmetros residem em `config.yaml`, fonte única de verdade: alterar um experimento significa alterar esse arquivo, não os notebooks. O módulo `src/config.py` carrega essa configuração e expõe `set_seeds()`, que fixa as sementes de `random`, `numpy` e `tensorflow` para reprodutibilidade.

4.3 Decisões de hiperparâmetros e proveniência

Um princípio metodológico central do projeto é a **rastreabilidade da proveniência** de cada hiperparâmetro. Cada escolha é classificada como: *fundamentada* (ancorada no enunciado do projeto ou na literatura), *default de literatura* (convenção da implementação de referência), *decisão de projeto* (escolha de *design* sem fonte direta, com *rationale* explícito) ou *provisória* (valor a ser calibrado por experimento). A Tabela 2 consolida as principais decisões; os valores provisórios não devem sustentar resultados finais sem validação.

4.4 Riscos metodológicos registrados

Três riscos foram documentados já nesta fase, para serem honrados na implementação: (i) o `validation_split` do Keras separa o *bloco final* dos dados antes de embaralhar — logo `shuffle=False` é obrigatório, sob pena de vazamento temporal; (ii) o limiar dinâmico deve ser *causal*, usando apenas observações passadas; (iii) o `MinMaxScaler` ajustado no treino faz com que extremos do teste possam exceder o intervalo $[0, 1]$ — comportamento esperado e desejável, pois é onde residem as anomalias, mas que torna o limiar sensível a *outliers* isolados (motivando o uso de percentil em vez do máximo).

Tabela 2: Principais hiperparâmetros e sua proveniência (consolidado dos ADRs).

Parâmetro	Valor	Proveniência
<code>window_size</code>	30	Fundamentada [4, 7]
<code>lstm_units</code>	64	Default de literatura [7]
<code>dropout</code>	0,2	Default de literatura [7]
<code>latent_dim</code>	16	Decisão de projeto (calibrar)
<code>loss</code>	MSE	Fundamentada [4]
<code>shuffle</code>	False	Fundamentada (causalidade) [7]
<code>validation_split</code>	0,1	Default de literatura [7]
<code>threshold_percentile</code>	95	Fundamentada (enunciado)
<code>dynamic_window</code>	252	Provisória (≈ 1 ano útil)
<code>n_injections</code>	50	Provisória

5 M1 — Coleta e Análise Exploratória

5.1 Coleta e cache

Os dados são obtidos via `yfinance` com `auto_adjust=True`, de modo que o preço de fechamento já incorpora ajustes por *splits* e dividendos. A arquitetura de dados separa estritamente o acesso à rede da leitura: as funções `cache_*` baixam e persistem os dados em `data/raw/` (CSV, versionado), enquanto `load_*` — usadas por todo o *pipeline* — leem apenas do cache, jamais da API. Isso garante execução *offline* e reprodutível dos notebooks.

5.2 Integridade das séries

As quatro séries apresentam **3724 pregões** cada (2010-01-04 a 2024-12-30), sem valores ausentes (NaN) após o ajuste. A Tabela 3 resume os achados. A função `integrity_report` computa, por ativo, contagens de NaN, dias com volume zero (halts/iliquidez) e os extremos de log-retorno diário, que funcionam como indicadores tanto de anomalias reais quanto de eventuais artefatos.

Tabela 3: Integridade e maior queda diária de log-retorno por ativo.

Ticker	Pregões	NaN	Vol. zero	Maior queda (data)
PETR4.SA	3724	0	54	−0,352 (2020-03-09)
VALE3.SA	3724	0	27	−0,282 (2019-01-28)
AMER3.SA	3724	0	25	−1,484 (2023-01-12)
ITUB4.SA	3724	0	34	−0,198 (2021-10-04)

5.3 Anomalias reais identificadas

A EDA já revela eventos extremos que servirão de âncora para a validação narrativa (M6):

- **PETR4**, 09/03/2020: queda associada ao início da pandemia de COVID-19 e à guerra de preços do petróleo.
- **VALE3**, 28/01/2019: reação ao rompimento da barragem de Brumadinho.
- **AMER3**, 12/01/2023: divulgação da fraude contábil da Americanas (queda de $\approx 78\%$ no dia).
- **ITUB4**, 04/10/2021: maior queda do setor financeiro no período.

A distribuição dos log-retornos exhibe caudas pesadas (leptocurtose), o que reforça a escolha de limiares por percentil em vez de critérios baseados em desvio-padrão.

5.4 O caso AMER3

A série da Americanas merecia atenção especial por causa da reorganização societária e do colapso pós-fraude. Dois pontos foram esclarecidos pela EDA:

1. **Histórico de treino existe.** Diferentemente da hipótese inicial de ausência de dados em 2010–2019, o `yfinance` fornece a série desde 2010 por continuidade de *ticker* (herdada da Lojas Americanas). Há, portanto, dados de “normalidade” para o ativo.
2. **Sinal vs. artefato.** A queda de 12/01/2023 é uma anomalia *real* e deve ser preservada (é justamente o evento-alvo). Já o salto de log-retorno $\approx +1,03$ em 14/11/2024 é compatível com um *grupamento* (*reverse split*) não totalmente ajustado pela fonte, configurando provável **artefato**.

Decidiu-se, para a etapa de pré-processamento (M2), **corrigir o fator de grupamento** do AMER3, em vez de recortar o período — preservando a continuidade temporal e o caso completo do ativo.

6 M2 — Pré-processamento

Esta etapa transforma as séries de preço na representação consumida pelo modelo, respeitando a ordem metodológica que evita vazamento temporal (Seção 3).

6.1 Log-retornos

A feature principal é o retorno logarítmico diário $r_t = \ln(P_t/P_{t-1})$, calculado sobre o fechamento ajustado. Por ser invariante a transformações multiplicativas de escala, é imune a fatores de *split*/grupamento já aplicados ao preço — propriedade relevante para o caso AMER3 (Seção 6.3).

6.2 Split temporal e normalização

Aplica-se primeiro o *split* temporal (treino até 2019; teste a partir de 2020) e só então a normalização: o `MinMaxScaler` é ajustado *exclusivamente* no treino e aplicado a ambas as partições. Como consequência esperada, uma fração dos pontos de teste extrapola o intervalo $[0, 1]$ — precisamente os extremos pós-2020 fora da faixa observada no treino. Esse comportamento é desejável: é onde se concentram as anomalias, e o erro de reconstrução tende a crescer ali. As janelas deslizantes (`window_size = 30`) são geradas *dentro* de cada partição, nunca cruzando a fronteira treino/teste. O resultado são ≈ 2450 janelas de treino e ≈ 1215 de teste por ativo, no formato $(n, 30, 1)$.

6.3 Revisão da hipótese sobre o AMER3

Em M1, levantou-se a hipótese de que o salto de log-retorno de $\approx +1,03$ em 14/11/2024 seria um *artefato* de grupamento, e decidiu-se, provisoriamente, “corrigir o fator”. A investigação conduzida em M2 **refutou essa hipótese** e ilustra a importância de validar premissas com os dados:

- Os grupamentos reais do AMER3 ocorreram em **julho e agosto de 2024** (quatro eventos de razão 1:100 registrados pela fonte) e **já são corrigidos por `auto_adjust=True`**: o log-retorno ajustado nessas datas é pequeno ($+0,09$; $+0,01$; $-0,18$; $+0,34$), com série contínua.
- Em 14/11/2024 o preço bruto saltou de R\$ 3,36 para R\$ 9,41 com volume cerca de três vezes a média e *sem split* registrado — caracterizando um **movimento de mercado real** (especulação durante a recuperação judicial), assim como o salto de 15/08/2024.
- Como o log-retorno é invariante a escala, *splits* já aplicados não produzem retorno espúrio.

Decisão revista: nenhuma correção é aplicada; o AMER3 segue o mesmo pipeline dos demais ativos, e seus saltos de 2024 são tratados como anomalias reais (verdade narrativa para M6). O registro completo está no ADR-0007.

7 Conclusões Parciais e Próximos Passos

Até este ponto, o projeto estabeleceu uma base reproduzível (M0), validou a qualidade dos dados (M1) e construiu a representação de entrada do modelo (M2). Os principais resultados parciais são:

- Infraestrutura com configuração centralizada, *seeds* fixas e proveniência documentada de cada hiperparâmetro, separando escolhas fundamentadas de valores provisórios.
- Quatro séries íntegras (3724 pregões, sem NaN), com anomalias reais já mapeadas como âncoras para a validação narrativa.
- Esclarecimento do caso AMER3: a hipótese de artefato de agrupamento foi refutada com os dados, e o ativo segue o pipeline padrão (Seção 6.3).
- Representação de entrada pronta: log-retornos normalizados em janelas de 30 passos, com split temporal e scaler restrito ao treino (sem vazamento).

Próximos passos (M3 — Modelo e Treino). Implementar e treinar o LSTM-Autoencoder (um modelo por ativo), com `EarlyStopping` e validação por bloco cronológico (`shuffle=False`), e conduzir o estudo de sensibilidade de `window_size` e `latent_dim`. As etapas subsequentes cobrirão os limiares de detecção (M4), a avaliação quantitativa por injeção sintética (M5) e a correlação com eventos e a comparação setorial (M6).

Registro de decisões. Todas as decisões de configuração estão documentadas como ADRs em `docs/adr/`. Este relatório é atualizado a cada fechamento de *milestone*, consolidando decisões, fontes, observações e conclusões para compor o artigo final.

Referências

- [1] Martín Abadi et al. TensorFlow: Large-scale machine learning on heterogeneous systems, 2015. <https://www.tensorflow.org/>.
- [2] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- [3] IEEE. Anomaly detection on Bitcoin values. In *Proceedings of the IEEE*, 2021.
- [4] Susan Li. Time series of price anomaly detection with LSTM. Towards Data Science, 2020. <https://medium.com/data-science/time-series-of-price-anomaly-detection-with-lstm-11a12ba4f6d9>.
- [5] Yifan Liu et al. Robust anomaly detection in financial markets using LSTM autoencoders and generative adversarial networks. *Journal of Economics, Finance and Accounting Studies*, 2025. OPAST Publishers.
- [6] Dejan Petrovic. Anomaly detection in stock price with LSTM autoencoder. GitHub, 2019. <https://github.com/dpetrovic89/Anomaly-Detection-in-Stock-Price-with-LSTM-Autoencoder>.
- [7] Venelin Valkov. Time series anomaly detection with LSTM autoencoders using Keras in Python. curiously.com, 2019. <https://curiously.com/posts/anomaly-detection-in-time-series-with-lstms-using-keras-in-python/>.